

Evidencia — Módulo de Control de Citas (HIS)

1. Levantar el entorno

```
docker compose up -d --build
```

```
Container his_citas_db      Started (healthy)
Container his_citas_app     Started
Container his_citas_web     Started
```

```
docker ps
```

NAMES	IMAGE	STATUS	PORTS
his_citas_web	nginx:1.27-alpine	Up	0.0.0.0:8080->80/tcp
his_citas_app	segundo-parcial-analisis-sistemas-2-app	Up	9000/tcp
his_citas_db	mysql:8.0	Up (healthy)	0.0.0.0:3306->3306/tcp

2. Migraciones y datos semilla

```
docker compose exec app php artisan migrate:fresh --seed
```

```
2024_01_01_000000_create_doctores_table ..... DONE
2024_01_01_000001_create_pacientes_table ..... DONE
2024_01_01_000002_create_citas_table ..... DONE
```

```
Database\Seeders\DoctorSeeder ..... DONE
Database\Seeders\PacienteSeeder ..... DONE
Database\Seeders\CitaSeeder ..... DONE
```

```
docker compose exec db mysql -uhis_user -phis_password his_citas \
-e "SELECT COUNT(*) FROM doctores; SELECT COUNT(*) FROM pacientes; SELECT COUNT(*) FROM citas;"
```

```
doctores: 5
pacientes: 7
citas: 6
```

3. API REST (curl)

Crear cita — 201

```
curl -s -w "\nHTTP %{http_code}\n" -X POST http://localhost:8080/api/citas \
-H "Content-Type: application/json" \
-d '{"paciente_id":1,"doctor_id":1,"fecha_inicio":"2026-09-25 09:00:00","fecha_fin":"2026-09-25 09:30:00"}
```

```
HTTP 201
```

Conflicto de horario con el mismo doctor — 409

```
curl -s -w "\nHTTP %{http_code}\n" -X POST http://localhost:8080/api/citas \
-H "Content-Type: application/json" \
-d '{"paciente_id":2,"doctor_id":1,"fecha_inicio":"2026-09-25 09:15:00","fecha_fin":"2026-09-25 09:45:00"}
```

```
{"message":"El doctor ya tiene una cita activa en ese horario."}
```

```
HTTP 409
```

Cita inexistente — 404

```
curl -s -o /dev/null -w "HTTP %{http_code}\n" http://localhost:8080/api/citas/9999
```

```
HTTP 404
```

Datos inválidos — 400

```
curl -s -w "\nHTTP %{http_code}\n" -X POST http://localhost:8080/api/citas \
-H "Content-Type: application/json" -d '{"paciente_id":999,"doctor_id":2,"fecha_inicio":"","motivo":"'
{"message":"The selected paciente id is invalid. (and 3 more errors)","errors":{...}}
HTTP 400
```

Reprogramar (drag & drop) — 200

```
curl -s -w "\nHTTP %{http_code}\n" -X PUT http://localhost:8080/api/citas/7 \
-H "Content-Type: application/json" \
-d '{"fecha_inicio":"2026-09-26 10:00:00","fecha_fin":"2026-09-26 10:30:00"}'
HTTP 200
```

Cambiar estado — 200 / transición inválida — 400

```
curl -s -w "\nHTTP %{http_code}\n" -X PATCH http://localhost:8080/api/citas/7/estado -d '{"estado":"con"
# HTTP 200
curl -s -w "\nHTTP %{http_code}\n" -X PATCH http://localhost:8080/api/citas/7/estado -d '{"estado":"can"
# HTTP 200
curl -s -w "\nHTTP %{http_code}\n" -X PATCH http://localhost:8080/api/citas/7/estado -d '{"estado":"con"
# {"message":"No se puede cambiar la cita de estado 'cancelada' a 'confirmada'."}
# HTTP 400
```

Listar / filtrar — 200

```
curl -s "http://localhost:8080/api/citas?doctor_id=1"
curl -s http://localhost:8080/api/doctores
curl -s http://localhost:8080/api/pacientes
```

4. Interfaz FullCalendar

Verificado manualmente en <http://localhost:8080/calendario>:

- Vista mensual y semanal (month / week) con eventos coloreados por estado (amarillo=pendiente, azul=confirmada, gris=cancelada, verde=atendida) y leyenda en el encabezado.
- Clic en un día vacío abre el modal “Nueva cita” con selects de paciente/doctor poblados desde `/api/doctores` y `/api/pacientes`; al guardar hace `POST /api/citas` y refresca el calendario.
- Clic en un evento abre el modal de detalle (paciente, doctor, horario, motivo, estado) con acciones para confirmar/atender/ cancelar, ocultando las transiciones no permitidas.
- Arrastrar un evento a otro día dispara `PUT /api/citas/{id}`; se comprobó que el cambio persiste en MySQL (`GET /api/citas/{id}` refleja la nueva fecha) y que, si el servidor responde 409, el evento vuelve a su posición original.
- Filtro por doctor sobre el propio listado de la API.
- Probado en resolución de escritorio (1280px) y tablet (768px), manteniéndose usable en ambas.

5. Historial Git — ramas, PRs y merges

```
git log --oneline --graph --all
```

```
* f15f772 feat(ui): integra FullCalendar interactivo consumiendo la API de citas
* 23ab761 Merge pull request #3 from Josuemart555/feature/validacion-conflictos-estados
|\
| * 5ed5de3 feat(citas): valida conflictos de horario y controla transición de estados
|/
* 23c5d54 Merge pull request #2 from Josuemart555/feature/api-rest-citas
```

```

|\
| * a116a67 feat(api): agrega API REST de citas, doctores y pacientes
|/
* 608ab4a Merge pull request #1 from Josuemart555/feature/docker-mysql-schema
|\
| * 5536bde feat(db): agrega esquema de citas médicas y datos semilla
| * f64593e feat(docker): agrega stack Docker con nginx, PHP-FPM 8.4 y MySQL 8
|/
* b3bc92a chore: scaffold base Laravel 11 project

```

Pull Requests (repositorio Josuemart555/segundo-parcial-analisis-sistemas-2):

#	Rama	Alcance	RQF/RQNF
#1	feature/docker-mysql-schema	Docker Compose + esquema MySQL + seeders	RQNF-01, RQNF-02
#2	feature/api-rest-citas	API REST de citas/doctores/pacientes	RQF-01, RQF-06, RQF-07, RQF-08, RQNF-03
#3	feature/validacion-conflictos-estados	Conflictos de estados + estados	RQF-03, RQF-05, RQNF-03, RQNF-07
#4	feature/fullcalendar	FullCalendar interactivo	RQF-02, RQF-04, RQF-09, RQF-10, RQNF-06
#5	feature/editar-citas	Edición completa de citas (paciente, doctor, horario, motivo) desde el calendario	RQF-01, RQF-03, RQNF-03, RQNF-07
#6	feature/usuarios-roles-permisos	Login (Breeze), roles/permisos (spatie), Doctor fusionado en User, CRUD de especialidades/estados de cita/usuarios	RQF-01, RQF-03, RQF-07, RQNF-03, RQNF-04, RQNF-05, RQNF-07

Las 6 ramas fueron fusionadas a main mediante Pull Request (merge commit), quedando el historial completo visible con `git log --graph --all`.

7. Login, roles/permisos y CRUDs de administración (rama feature/usuarios-roles-permisos)

- Login con Laravel Breeze (Blade) + Sanctum stateful; todo el sistema (calendario, API, CRUDs) requiere sesión iniciada.
- Roles con `spatie/laravel-permission`: admin, recepcionista, doctor.
- Doctor fusionado dentro de User (rol doctor + `especialidad_id`); se eliminó la tabla `doctores`.
- CRUD de especialidades (antes texto libre) y de estados de cita (antes un enum fijo en código, ahora con `color/es_terminal/bloquea_horario` administrables).
- CRUD de usuarios con asignación de rol y, si es doctor, especialidad.
- El calendario construye su leyenda y los botones de cambio de estado dinámicamente desde `GET /api/estados-cita`.

Evidencia

```

# Sin sesión, /calendario redirige a login
curl -sI http://localhost:8080/calendario | head -1
# HTTP/1.1 302 Found

# Conflicto de horario con el nuevo esquema (users + estados_cita)
POST /api/citas (doctor válido, horario libre) -> HTTP 201, "estado":"pendiente"

```

```
POST /api/citas (mismo doctor, horario solapado) -> HTTP 409
POST /api/citas (doctor_id de un usuario admin) -> HTTP 400 "The selected doctor id is invalid."
```

```
# Usuario sin rol admin
GET /usuarios -> 403 Forbidden
```

```
# Tests automatizados (sqlite en memoria, nunca contra la MySQL de Docker)
docker compose exec app php artisan test
# Tests: 31 passed (71 assertions)
```

Verificado en el navegador: login como `admin@his.test` / `password` (ve Calendario + Especialidades + Estados + Usuarios), y como `carlos.perez@his.test` / `password` (rol doctor, solo ve Calendario y recibe 403 en `/usuarios`). Se creó una especialidad, un estado de cita y un usuario con rol doctor desde sus respectivos CRUDs, confirmando que el nuevo doctor aparece de inmediato en el selector del calendario.

6. Edición completa de citas (rama `feature/editar-citas`)

El endpoint `PUT /api/citas/{id}` ya validaba conflictos de horario en el servidor excluyendo la propia cita; esta rama expone la edición completa (paciente, doctor, fecha/hora, motivo) desde el modal de detalle del calendario y exige los 5 campos en `UpdateCitaRequest` (antes `paciente_id`/ `doctor_id` eran opcionales).

Evidencia (curl)

```
# Edición completa válida
curl -s -w "\nHTTP %{http_code}\n" -X PUT http://localhost:8080/api/citas/1 \
  -H "Content-Type: application/json" \
  -d '{"paciente_id":2,"doctor_id":3,"fecha_inicio":"2026-09-28 08:00:00","fecha_fin":"2026-09-28 08:30:00","motivo":"E"}'
# HTTP 200

# Falta un campo requerido (paciente_id)
curl -s -w "\nHTTP %{http_code}\n" -X PUT http://localhost:8080/api/citas/1 \
  -H "Content-Type: application/json" \
  -d '{"doctor_id":3,"fecha_inicio":"2026-09-28 08:00:00","fecha_fin":"2026-09-28 08:30:00","motivo":"E"}'
# {"message":"The paciente id field is required.", ...}
# HTTP 400

# Edición hacia un horario que choca con otra cita activa del mismo doctor
curl -s -w "\nHTTP %{http_code}\n" -X PUT http://localhost:8080/api/citas/1 \
  -H "Content-Type: application/json" \
  -d '{"paciente_id":1,"doctor_id":2,"fecha_inicio":"2026-09-20 10:30:00","fecha_fin":"2026-09-20 11:00:00","motivo":"E"}'
# {"message":"El doctor ya tiene una cita activa en ese horario."}
# HTTP 409
```

Evidencia (UI)

Verificado manualmente en `http://localhost:8080/calendario`:

- Detalle de la cita → botón “Editar” (oculto si el estado es terminal: cancelada/atendida) → abre el modal “Editar cita” con paciente, doctor, fecha/hora y motivo prellenados con los datos actuales.
- Al guardar con datos válidos: `PUT /api/citas/{id}`, el modal se cierra y el evento se actualiza en el calendario; confirmado también vía `GET /api/citas/{id}` que el cambio persistió en MySQL.
- Al guardar con un horario en conflicto: el modal permanece abierto y muestra el mensaje de error inline (sin perder los datos ya escritos).